Graphs Deserve Some Attention

Patrick R. Nicolas - Original Substack Article

Target audience: Intermediate
Estimated reading time: 9'

Follow on LinkedIn

Graph Attention Models offer a hybrid solution to the limitations of spectral and spatial graph processing addressing shortcoming of graph convolutional networks (fixed weight and transductive learning strategy) and graphSAGE (variable size of node neighborhood).

Newsletter
Hands-on Geometric Deep Learning

GitHub
Repositories

Articles
Geometric Learning with Python

Blog
Practical Machine Learning

What you will learn: How to use PyTorch Geometric to implement a GAT while leverage reusable neural blocks.

Table of Contents

🎯 Why this Matters 2
🎨 Modeling & Design Principles 3
Overview 3
GAT vs. GCN 3
GAT vs. GraphSAGE 3
Graph Attention Model 4
⚙️ Hands-on with Python 6
Environment 6
Torch Attention Modules 6
Internal Attention Function 6
Class Design 7
Graph Neural Blocks 7
Graph Neural Models 8
🔎 Implementation 8
📈 Evaluation 10
📘 References 13

💎 For the original, complete article on Substack
Original Article

🎯 Why this Matters

Purpose: Graph Attention Networks (GATs) overcome the traditional constraints found in both spectral-based models and spatial inductive methods for sequences. Unlike GCNs, which often rely on static or degree-based weights, GATs employ a dynamic attention mechanism to optimize node embeddings. Additionally, by accommodating variable neighborhood sizes, GATs offer superior data retention over GraphSAGE's neighborhood sampling approach.

Audience: Data scientists and engineers developing complex, non-linear models using graph representation.

Our focus is the practical design and implementation of Geometric Deep Learning. Modeling & Design Principles will resonate with scientists, while Hands-on with Python targets software engineers.

🔎 **clarify the GitHub code and can be skipped.**

💎 **Premium content**

🎨 Modeling & Design Principles

Overview

GATs were introduced in 2018 [ref *6*] to address the shortcoming of both Convolutional Networks (CNNs) and Graph Convolutional Networks (GCNs)

My evaluation of GATs follows my articles on Graph Sampling [ref *1*, *2*] Graph Convolutional Networks (GCN) [ref *3*, *4*] and Graph SAGE [ref *5*].

Here is a comparative table for GCN, GraphSAGE and GAT.

Features	GCN	GraphSAGE	GAT
Learning strategy	Transductive	Inductive	Inductive
Interpretability	Low	Low	High
Scalablity	Low	High	Medium
Learning scope	Entire graph	Samples	Local nodes
Message aggregation	Static	Samples	Dynamic
Aggregation criteria	Node degree	Uniform dist.	Attention

Table 1 Comparison of GAT, GCN and GraphSAGE models

GAT vs. GCN

GAT uses learnable attention weights assigned to different neighboring nodes based on their features while GCN assigns a fixed weight to neighbors. Moreover, GAT have an inductive learning strategy which does not require loading the entire graph as with GCN.

GAT vs. GraphSAGE

GraphSAGE samples a fixed number of neighbors that may cause loss of valuable information, while GAT process neighbors of any size each with a unique weight.

Graph Attention Models were introduced to address the limitation of both Spectral representation of graph (Laplacian) and Spatial induction (convolution on graph) for sequence-based tasks. The basic idea is to compute the hidden representation of each node from its neighbors using self-attention strategy. It is also directly applicable to inductive learning problems - generalization over unseen graph nodes (e.g., GraphSAGE) [ref *6*].

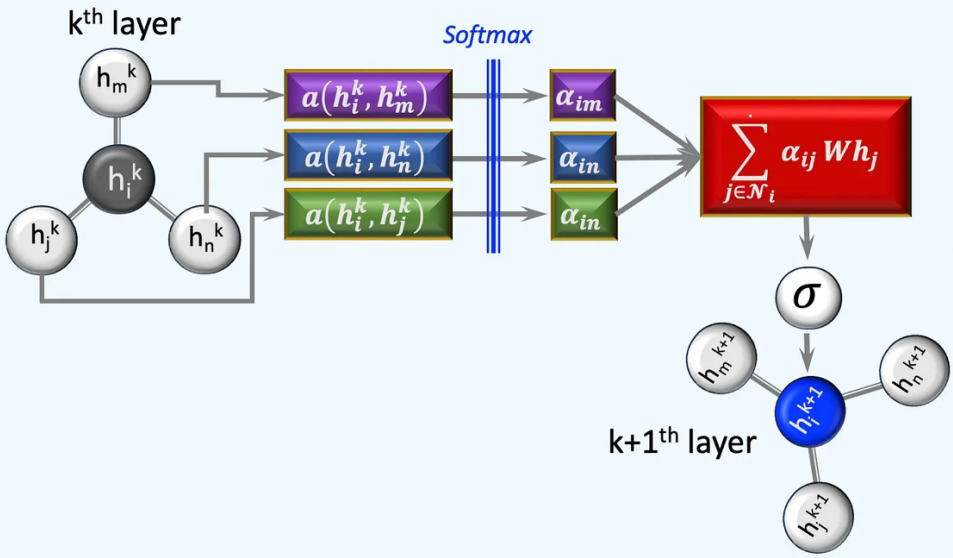

Fig. 1 Illustration of the transformation of graph nodes features in kth layer of a graph attention network

Graph Attention Model

🤖 For math-minded readers

Generic Message Passing and Aggregation

$$z_u^{k+1} = \Phi\left(\Lambda\left(z_u^k, \bigoplus_{j \in \mathcal{N}_i} \mu_{ij}\right)\right) \qquad Update\ k+1\ message\ passing\ layer$$

$$\Phi : \mathbb{R}^{l_k} \rightarrow \mathbb{R}^{l_{k+1}} \qquad Differential,\ trainable\ layer$$

$$\bigoplus_{j \in \mathcal{N}_i} \qquad Permutation\ invariant\ aggregation$$

$$\mu_{ij} \qquad Feature\ vector\ interaction\ nodes\ i,\ j$$

$$\Lambda \qquad Interaction\ nodes\ in\ aggregation$$

Graph Attention

$$\mu_{ij} = a\left(h_i^k, h_j^k\right).\psi\left(h_j^k\right) \qquad Feature\ vec.\ interaction\ nodes\ i,\ j$$

$$\psi\left(h_j^k\right) = W.h_j^k + b \qquad Affine\ linear\ transformation$$

$$W \qquad Matrix\ unlearned\ weights$$

$$a\left(h_i^k, h_j^k\right) = \sigma\left(a^T.\left(h_j^k || h_m^k\right)\right) \qquad Attention\ function,\ \ \sigma : Activation$$

$$\alpha_{ij} = softmax(a\left(h_i^k, h_j^k\right)) \qquad Softmax\ normalization$$

$$h_i = \sigma\left(\sum_{j \in \mathcal{N}_j} \alpha_{ij} W h_j\right) \qquad Update\ node\ features\ with\ attention\ \alpha$$

Notes:

- The symbol || represents the concatenation
- The parameters a and W are learned

📌 *Transformers can be constructed as GATs. Transformer can be interpreted as a fully connected graph where token embeddings act as nodes. Each node can attend to every other node through attention connections, enabling unrestricted information flow.*

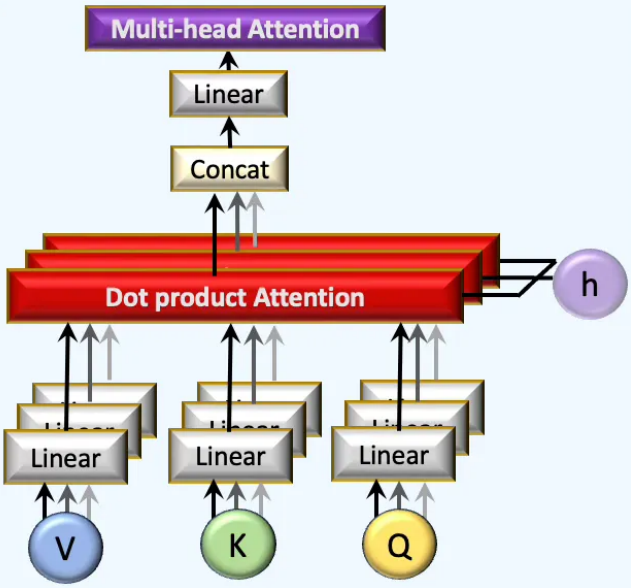

Fig. 2 Illustration of Transformers as Graph Attention Models

*The outer product - query (**Q**) key (**K**) vectors - generates a message between nodes, and the multiplication with the value (**V**) vector corresponds to message aggregation and node state update.*

📌 *GAT is a **discretization of the graph diffusion**.*
*The diffusion is defined by the heat equation. Given a latent variable **z** and the i th node with positional value **u** and feature **x**.*
🤖 For math-minded readers

$$z_i = (u_i, x_i) \qquad \frac{\partial z_i(t)}{\partial t} = \alpha \Delta_V z_i(t)$$

Applied to graph given two nodes *i* and *j*

$$\frac{\partial z_i(t)}{\partial t} = \sum_{(i,j)\in E} \alpha\left(z_i(t), z_j(t)\right)\left(z_j(t) - z_i(t)\right))$$

Applying the Euler discretization formula, at step *k* with an interval *tau*

$$z_i^{k+1} = \sum_{i,j\in E} \alpha\left(z_i^k, z_j^k\right) z_i^k$$

⚙️ Hands-on with Python

Environment

- Libraries: *Python 3.12.5, Numpy 2.3.5, PyTorch 2.7.2 PyTorch Geometric 2.6.1*
- Implementation code**:** *geometriclearning/python/deeplearning/model/graph/graph_attention_model.py*
- Evaluation code: *geometriclearning/play/graph_attention_model_play.py*
- The source tree is organized as follows: features in python/, unit tests in **tests/,** and newsletter evaluation code in play/.
- To enhance the readability of the algorithm implementations, we have omitted non-essential code elements like error checking, comments, exceptions, validation of class and method arguments, scoping qualifiers, and import statements.

Torch Attention Modules

- GATConv: Basic linear layer applied after concatenating features. It uses static attention (where the ranking of importance between neighbors is the same for all query nodes
- GATv2Conv: Contrary to GATConv , GATv2Conv implements dynamic attention, applicable if your model seems to be smoothing features too much
- TransformerConv: It implements a multi-head dot-product attention mechanism instead of the additive attention used in GATConv. It only computes the transformed and aggregated node features, scheme particularly beneficial if you want to leverage the gated residual connections.
- RGATConv: It learns separate attention weights for different relation types of heterogeneous graphs. This module is particularly suitable for complex knowledge graphs and datasets with multi-type edges.
- AGNNConv: Suitable for semi-supervised learning as it focuses on attentional propagation without a learnable linear transformation in every layer.
- FusedGATConv: An extension to the basic GATConv layer that fuses the message-passing operations into a single kernel to reduce memory footprint.
- GPSConv: A hybrid, wrapper-style module local message-passing layer-global attention layer.

📌 *As a general rule, graph attention layers do not need a pooling mechanism for node classification and link prediction as the attention mechanism already pools information from its neighbors. However, graph classification may benefit from pooling.*

Internal Attention Function

The following PyTorch attention modules, GATConv, GATv2Conv, RGATConv and FusedGATConv have a default, internal activation module, LeakyReLU with a negative slope 0.2. Therefore, there is no need for developer to specify an activation function.

Being a wrapper layer the GPSConv has a default ReLU activation function.

Finally, TransformerConv and AGNNConv may need a user-specified activation function depending on the application.

Class Design

Reusable neural blocks and model builders are essential for creating modular, configurable graph neural networks. Let's take a closer look at these concepts.

Graph Neural Blocks

Reusable neural blocks, introduced in [ref *6*], provide a simple and convenient way to package related PyTorch components into modular units that can be dynamically assembled for efficient model development.

In our framework, a graph neural block typically includes a message passing and aggregation module, along with optional PyTorch layers such as activation, batch normalization, and dropout.

This base class organizes four core components:

- message_passing_module: the message passing operator as torch module
- batch_norm_module: optional batch normalization
- activation_module: optional activation function (e.g., ReLU, LeakyReLU)
- dropout_module: optional node-level dropout regularization.

The following class diagram, *Figure 3*, illustrates the relation between the various block and torch modules.

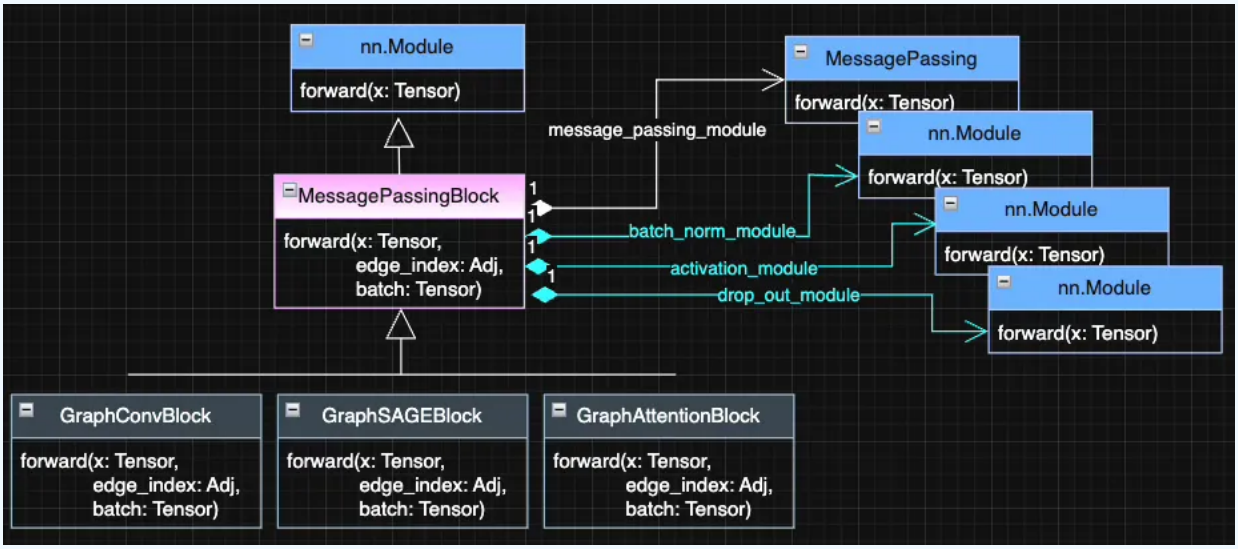

Fig. 1 Class diagram for Graph Convolutional, Attention and SAGE neural blocks

Graph Neural Models

All graph neural network model classes inherit from the abstract base class *GraphBaseModel*. They share a common training and validation method, *train_model,* which relies on the utility class *GNNTraining* introduced in a previous article [ref *8*].

The forward method can be overridden in specific models. It iteratively applies message-passing blocks (derived from *MessagePassingBlock*) along with fully connected layers implemented as instances of *MLPBlock*.

The relationship between the various class-models and their component is illustrated in Figure 4.

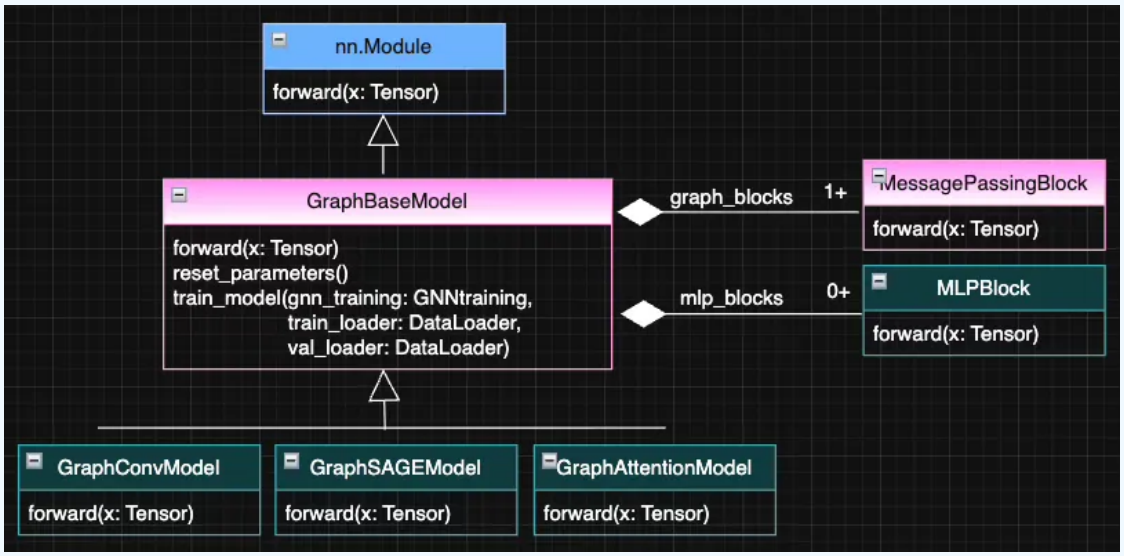

Fig. 4 Class diagram for Graph Convolutional, SAGE and attention models

A builder design pattern is used to instantiate a graph attention model using the class builder *GraphAttentionBuilder*. The builder generates a model from a set JSON attributes, *model_attributes*.

Fig. 5 Class diagram for Graph Attention Builder

🔎 Implementation

Following the same process as in previous articles for Graph Convolutional and *GraphSAGE* networks, let's define the class *GraphAttentionBlock* that wraps

- *attention_layer*: torch attention module
- *batch_norm_module*: optional batch normalization
- *activation_module*: optional activation function (e.g., ReLU, LeakyReLU)
- *dropout_module:* optional node-level dropout regularization.

The class *GraphAttentionBlock* inherits from the generic class *MessagePassingBlock* that generates the sequence of modules, *modules_list.*

```
# -------------- Code Snippet 1 --------------------

GATL = TypeVar('GATL')


class GraphAttentionBlock(MessagePassingBlock, Generic[GATL]):
    def __init__(self,
                 block_id: AnyStr,
                 attention_layer: GATL,
                 batch_norm_module: Optional[BatchNorm] = None,
                 activation_module: Optional[nn.Module] = None,
                 dropout_module: Optional[nn.Dropout] = None) -> None:

        GraphAttentionBlock.__validate(attention_layer, activation_module, batch_norm_module)
        super(GraphAttentionBlock, self).__init__(block_id,
                                                  attention_layer,
                                                  batch_norm_module,
                                                  activation_module,
                                                  dropout_module)
    def forward(self,
                x: torch.Tensor,
                edge_index: Adj,
                batch: torch.Tensor = None) -> torch.Tensor:
```

The static method *__validate* raises an exception if the type of attention module is not supported or if the *activation_module* is specified for torch activation modules that have already an internal activation function.

Attention module	Activation module	Exception
GATConv, GATv2Conv RGATConv, FusedGATConv	Provided	ValueError
TransformerConv, AGNNConv	Not Provided	ValueError
Other type module		TypeError

Table 1. Exceptions raised for inconsistent attributes of a Graph Attention Block

The *forward* method invokes the dunder method, *__call__* for each torch modules (modules_*list*) registered by the constructor. PyTorch geometric *forward* method is invoked for the attention module with the adjacency matrix, *edge_index*.

```
# -------------- Code Snippet 2 --------------------

def forward(self,
            x: torch.Tensor,
            edge_index: Adj,
            batch: torch.Tensor = None) -> torch.Tensor:

    # Process all the torch modules if defined
    for module in self.modules_list:
        # Invokes the forward method with edge indices for the convolutional module
        if isinstance(module, (GATConv, GATv2Conv, TransformerConv, RGATConv,
                               AGNNConv, FusedGATConv, GPSConv)):
            x = module(x, edge_index)

        # Invokes the forward method with edge indices and batch for optional batch
        # normalization, activation and dropout ( regularization) modules
        else:
            x = module(x)
    return x
```

Next, we implement the class *GraphAttentionModel* as a container for

- *graph_attention_blocks:* Tuple of graph attention blocks
- *mlp_blocks*: Tuple of fully connected multi-layer perceptrons.

```
# -------------- Code Snippet 3 --------------------

GAT = TypeVar("GAT")

class GraphAttentionModel(GraphBaseModel, Generic[GAT]):
    def __init__(self,
                 model_id: AnyStr,
                 graph_attention_blocks: Tuple[GraphAttentionBlock[GAT]],
                 mlp_blocks: Optional[Tuple[MLPBlock]] = None) -> None:
        super(GraphAttentionModel, self).__init__(model_id,
                                                  graph_attention_blocks,
                                                  mlp_blocks)

    def forward(self, data: Data) -> torch.Tensor:

    def train_model(self,
                    training: GNNTraining,
                    train_loader: DataLoader,
                    val_loader: DataLoader) -> None:
```

The *forward* method for the graph attention model, iterates across all the attention block *graph_blocks*, then across all fully connected blocks, *mlp_blocks*.

```
# -------------- Code Snippet 4 --------------------

def forward(self, data: Data) -> torch.Tensor:
    # Step 1: Initiate the graph embedding vector
    x = data.x
    edge_index = data.edge_index

    # Step 2: Process forward the convolutional layers
    # Create and collect the output of each GNN layer
    for graph_attention_block in self.graph_blocks:
        # Implicit invoke forward method for the block
        x = graph_attention_block(x, edge_index, data.batch)

    # Step 3: Process the fully connected, MLP layers
    for mlp_block in self.mlp_blocks:
        # Invoke the forward method for the MLP block
        x = mlp_block(x)
    return x
```

Evaluation

Our configuration for the model, loader, and training attributes follows the established JSON template used for GCN [ref *4*] and GraphSAGE [ref *5*]. Consistent with our previous methodology, we utilize the NeighborLoader [ref *1*] for data sampling.

```
loader_attributes = {
    'id': 'NeighborLoader',
    'num_neighbors': neighbors,
    'batch_size': 32,
    'replace': True,
    'num_workers': 4
}
```

Our simple model consists of a single graph attention model without batch normalization and a single fully connected layer.

```
model_attributes = {
  'model_id': 'Graph Attention - Sampling [6, 3] - Cora,
  'graph_attention_blocks': [
    {
      'block_id': 'GAT_1',
      'attention_layer': GATConv(in_channels=_data.num_node_features,
                                 out_channels=64,
                                 heads=4),
      'num_channels': 64,
      'activation': None,
      'batch_norm': None,
      'dropout': 0.25
    }
  ],
  'mlp_blocks': [
    {
      'block_id': 'Output',
      'in_features': 64*4,
      'out_features': dataset.num_classes,
      'activation': None
    }
  ]
}
```

The dataset is loaded using one of the data source *Planetoid* predefined in PyTorch Geometric. We do not assume that the data set is perfectly balanced. This implementation relies on the *GraphDataLoader* class introduced in the article Demystifying Graph Sampling & Walk Methods [ref 7].

```
# --------------- Code Snippet 5 --------------------

# 1. Initialize the class weights for unbalanced data
class_weights = GraphDataLoader.class_weights(dataset[0])

# 2. Instantiate a graph attention model from the JSON attributes
graph_attention_builder = GraphAttentionBuilder(model_attributes)
graph_attention_model = graph_attention_builder.build()

# 3. Load one of the PyTorch Geometric Dataset - training and evaluation
graph_data_loader = GraphDataLoader(dataset_name=dataset_name,
                                    sampling_attributes=loader_attributes)
train_loader, eval_loader = graph_data_loader()

# 4. Setup and execute the training/evaluation of the model
trainer = GNNTraining.build(training_attributes)
graph_attention_model.train_model(trainer, train_loader, eval_loader)
```

The training is performed by the method *GNNTraining.train_model* introduced and described in a previous article, Plug & Play Training for Graph Convolutional Networks [ref *8*].

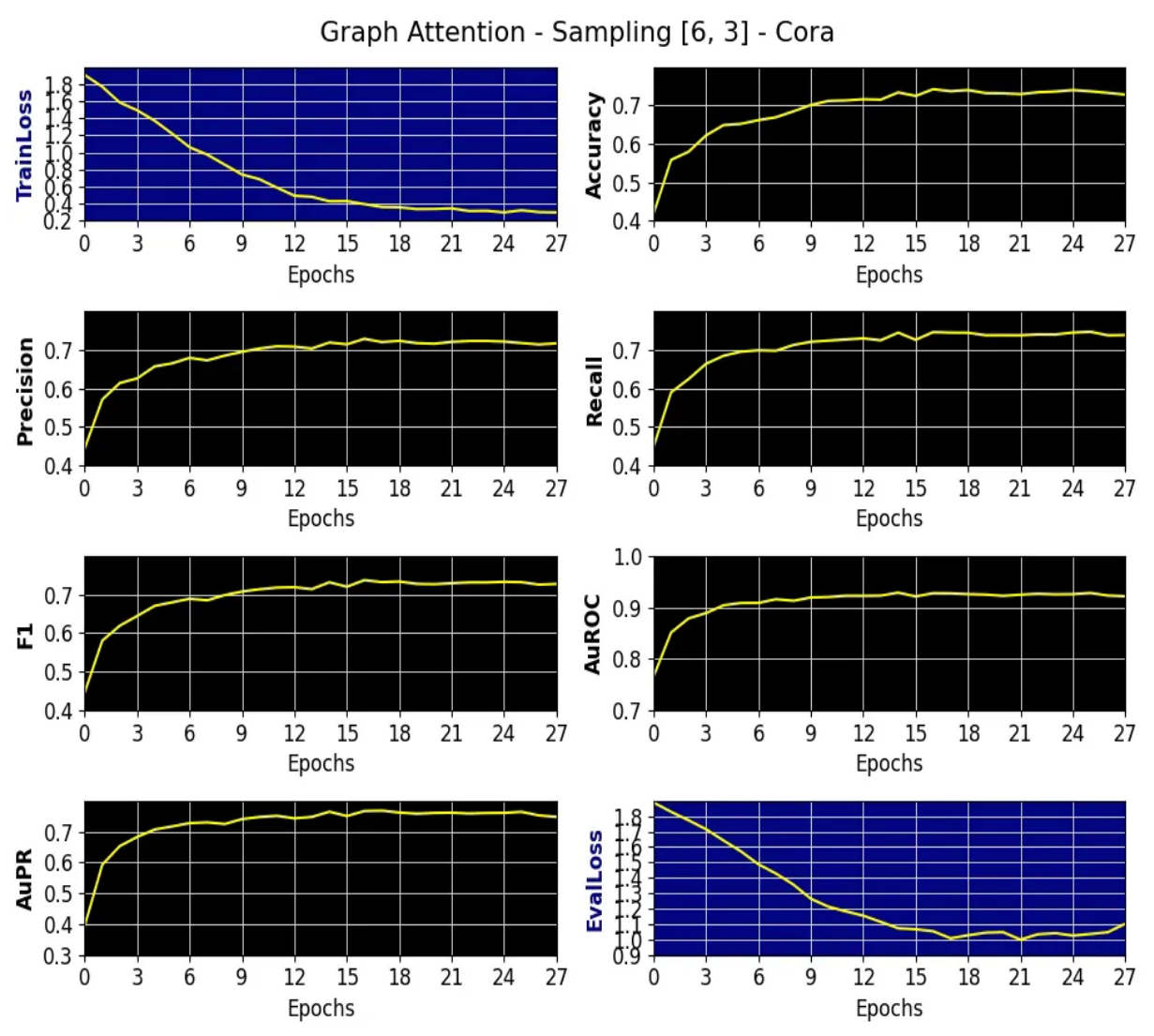

Fig. 6 Performance metrics for the Graph Attention Model using GATConv layer with neighborhood distribution (6, 3) on Cora data set

References

1. Demystifying Graph Sampling & Walk Methods - Hands-on Geometric Deep Learning, 2025
2. Neighbors Matter: How Homophily Shapes Graph Neural Networks - Hands-on Geometric Deep Learning, 2025
3. Taming PyTorch Geometric for Graph Neural Networks - Hands-on Geometric Deep Learning, 2025
4. Graph Convolutional or SAGE Networks? Shootout - Hands-on Geometric Deep Learning, 2025
5. Revisiting Inductive Graph Neural Networks - Hands-on Geometric Deep Learning, 2025
6. Graph Attention Networks P. Velickovic, G, Cucurull, A. Casanova, A. Romero, P. Lio, Y. Bengio, 2018
7. Demystifying Graph Sampling & Walk Methods - Graph Loaders - Hands-on Geometric Deep Learning, 2025
8. Plug & Play Training for Graph Convolutional Networks - Setting up training - Hands-on Geometric Deep Learning, 2025
9. *Design Patterns - Elements of Reusable Object-Oriented Software - Creational Pattern/Builder* . Gamma, R. Helm, R. Johnson, J. Vlissides - Addison-Wesley, 1995

💎 Key takeaways, exercises and paper review is available in the original Substack article
Graph Deserve Some Attention

Patrick Nicolas has over 30 years of experience in software and data engineering, architecture design and end-to-end deployment and support with extensive knowledge in machine learning. He has been director of data engineering at Aideo Technologies since 2017 and he is the author of "Scala for Machine Learning", Packt Publishing ISBN 978-1-78712-238-3 and *Geometric Learning in Python* newsletter and *Hands-on Geometric Deep Learning* blog